Parth.ai — Developer Portfolio

> **Live Site:** https://parth-singh-debugs.vercel.app
> **Built by:** Parth Singh · Senior Software Engineer · GEP Worldwide · IIIT Delhi '22

🚀 Quick Start

1. Get Your Free Groq API Key

All AI features run on **Groq's inference API** using **Llama 3.3 70B**.

1. Visit → **https://console.groq.com/keys**
2. Sign up for free (no credit card needed)
3. Click **"Create API Key"**
4. Copy your key (starts with `gsk_`)
5. Open any AI tool on the site and paste your key when prompted

> Your key is saved **only in your own browser's localStorage** — it never touches the repository or any server. Only you have access to it.

2. Deployment

- **Hosting:** Vercel — https://vercel.com/parth-singhs-projects-aa32ea42
- **Project Name:** `parth-singh-debugs`
- **Stack:** Pure HTML/CSS/JS — no build step, no dependencies
- **Deploy:** Push to GitHub → Vercel auto-deploys instantly

🤖 How the AI Works

All three AI tools call Groq's API directly from your browser using your own key. There is no backend server, no middleman, no data stored anywhere.

```
Your Browser
    │
    ├── Your Groq key (stored in your localStorage only)
    │
    └── fetch() → https://api.groq.com/v1/chat/completions
                      │
                      └── Llama 3.3 70B (Groq Cloud)
                              │
                              └── Response rendered in UI
```

Core concepts used across all three projects:

Concept	What it means here
LLM	Llama 3.3 70B — the model that powers all three tools
Prompt Engineering	Carefully written instructions that tell the model exactly what to do
JSON Mode	Forcing the model to return structured data so the UI can render it reliably
Generative UI	The model's output builds the interface — diagrams, tables, cards — at runtime
Adaptive AI	Your performance history shapes what questions you get next
Zero-shot Evaluation	The model judges your code without any pre-written test cases
Runtime Inference	Every request is a fresh live call — no pre-cached or hardcoded answers

---

📦 Projects

🔍 1. AI Code Review & Refactoring (`code-review.html`)

Paste any code — Python, JavaScript, C#, Java, Go — and get a full review with health scores, bugs, security issues, and a refactored version. Optionally adds inline comments to every function and line.

How it uses AI

The model is given a role — senior software engineer — and asked to review the code across specific areas: reusability, performance, security, naming, and SOLID principles. The focus areas

change based on what checkboxes you tick, so the prompt is built dynamically at runtime.

The model is instructed to return a strict JSON structure:

```json
{
  "scores": { "overall": 72, "reusability": 65, "performance": 80, "security": 70 },
  "summary": "...",
  "critical": [{ "title": "...", "description": "...", "fix": "..." }],
  "refactored": "corrected code here",
  "commented_code": "code with comments on every line",
  "positives": ["..."]
}
```

This structured output is what allows the UI to render score bars, colour-coded issue cards, and tabbed code panels — all built from the model's response at runtime. None of this UI exists before the model replies.

When "Add Code Comments" is selected, the prompt includes an additional instruction asking the model to annotate every function, class, and complex line with a plain-English explanation. The prompt literally changes based on your selections.

Key techniques: prompt engineering, structured JSON output, generative UI, conditional prompt construction, runtime inference

---

🏗️ 2. AI System Design Generator (`system-design.html`)

Type any app name — Uber, Netflix, WhatsApp — and get a full system design: SVG architecture diagram, free vs paid stack comparison across 9 layers, a 4-phase build roadmap, and scalability tradeoffs.

How it uses AI

The model is given a scale target (Startup / Mid-Scale / Hyper-Scale) alongside your app description. The scale context changes the entire output — a startup gets a simple monolith recommendation, a hyper-scale system gets Kubernetes, Kafka, and multi-region CDN. Same app, very different architecture depending on context.

The model returns a deeply nested JSON schema covering:

- Component names, descriptions, technologies, and layer assignments
- Connection edges between components (used to draw the SVG)

- A 9-layer stack comparison with free options, paid options, recommended choice, and reasoning
- A 4-phase roadmap with timelines and concrete milestones
- Tradeoffs and a scalability strategy paragraph

The SVG architecture diagram is generated entirely at runtime — the model outputs component names and connections, the frontend computes positions and draws the diagram. There is no static image anywhere.

Key techniques: context-aware generation, structured JSON output, generative SVG diagrams, domain reasoning across the full cloud stack, runtime inference

---

🎓 3. AI Adaptive Coding Tutor (`ai-tutor.html`)

150+ DSA questions across 7 topics (Arrays, Two Pointers, Sliding Window, Stack, Binary Search, Linked Lists, Dynamic Programming). The AI checks your code in real time, tracks where you struggle, and automatically focuses more questions on your weak areas.

How it uses AI

Code Evaluation: Every time you submit code, it is sent to the model along with the problem statement and a reference solution. The model judges whether your logic is correct — there are no pre-written test cases. It reasons about your approach, handles edge cases in its evaluation, and returns either a ✅ pass or ❌ fail with specific feedback.

```
Problem: "Two Sum". Example: [2,7,11,15], target=9 → [0,1]
Reference: def twoSum(nums, target): ...
Your code: [what you wrote]

Is the logic correct? Start with ✅ or ❌. Then give 1-2 sentences of feedback.
```

Adaptive Engine: Your performance on each topic is tracked in memory — correct answers, wrong answers, time taken. Topics where you struggle get a lower score, and the question queue is rebuilt every 5 questions to weight weak topics more heavily. So the tutor gets harder in your weak spots and easier where you're confident.

Personalised Hints: Every chat message includes your current problem, your code, and a summary of your weak topics. The model uses this to give advice that's specific to you — not generic tips.

Stuck Detection: If you haven't typed for 40+ seconds, a hint prompt appears automatically. The tutor notices when you're stuck before you ask.

Key techniques: zero-shot code evaluation, adaptive learning loop, personalised prompt context, behavioural signal detection, runtime inference

---

🎮 Games

5 games built directly into the portfolio with no external libraries:

Game	Notes
🦕 Dino Runner	Canvas game loop, progressive difficulty, collision detection
🐍 Snake	Grid state machine, mobile swipe support
🚀 Space Shooter	Asteroid physics, bullet collision
🧱 Breakout	Ball physics, paddle angle reflection
🃏 Memory Cards	CSS 3D card flip animations

All scores save to your browser's localStorage with a leaderboard.

🧱 File Structure

```
parth-ai-portfolio/
├── index.html          # Main portfolio
├── ai-tutor.html       # Adaptive coding tutor
├── code-review.html    # AI code review
├── system-design.html  # System design generator
├── parth.jpg           # Profile photo
└── resume.pdf          # Resume download
```

No build step. No npm. No frameworks. No backend. Everything runs in the browser.

👤 About

Parth Singh — Senior Software Engineer at GEP Worldwide
🎓 B.Tech CSE · IIIT Delhi 2022

🏆 GEP Kudos Award × 3
✈️ On-site deployment lead · JAPFA Go-Live · Indonesia
🌐 Clients: Bank of America · Kellogg's · Coca-Cola · Chevron
🔧 Python · C# .NET Core · Camunda BPM · Kafka · MongoDB · Azure

📧 parthsinghaug2000@gmail.com
🔗 linkedin.com/in/parth18356
🐙 github.com/parth18356

Pure HTML. Powered by Groq. Hosted on Vercel.